

FinVest Implementation Explanation

Prepared from the current project state in `D:\FinVest\backend` . This document is intended for teammates preparing for evaluation and viva-style questions.

Important context for teammates: this explanation is based on the user stories and module list shared during development, and on the latest code currently running in the project. Where the code slightly differs from the original wording of a user story, that difference is pointed out clearly so the team can answer evaluator questions honestly.

Project Overview

FinVest is a Flask-based personal finance and budgeting web application. The backend uses Flask, SQLAlchemy, MySQL, and MongoDB. MySQL stores the main transactional data such as users, settings, accounts, categories, transactions, budgets, goals, notifications, recurring transactions, and active sessions. MongoDB is used as a secondary store for activity logs and report snapshots.

The overall backend entry and registration flow is controlled from `backend/main.py` and `backend/app/__init__.py` . The main app factory registers blueprints for authentication, dashboard, budgets, goals, categories, notifications, reports, and transactions. It also injects display currency, language, notification count, and request timing into the template layer.

Part A: User Story Wise Explanation

US-1: Register and log in so the user can access the finance dashboard

Where implemented: `backend/app/routes/auth_routes.py` , `backend/app/templates/auth/auth.html` , `backend/app/static/css/auth.css` , `backend/app/services/admin_service.py` .

How it works: The auth blueprint exposes `/auth` , `/register` , `/login` , and `/logout` . Registration validates username pattern, email uniqueness, password strength, and recovery answers. Passwords are stored using Werkzeug password hashing. Login checks email, password, account active status, creates a Flask session, and stores a session token for active-session tracking. Admins are redirected to the admin dashboard and normal users to the main home dashboard.

Important viva note: the original story mentioned reset by token link and CSRF. The current code uses a stronger password policy and a recovery-question based reset flow instead of an email token link. So if asked, the team should say the project implements secure password recovery using security questions, not mail-based token reset.

US-2: Set currency, locale, and month-start

Where implemented: `backend/app/routes/dashboard_routes.py` in `account()` and `update_settings()` , model `backend/app/models/user_settings_model.py` , view `backend/app/templates/dashboard/account.html` , helper services `currency_service.py` and `i18n_service.py` .

How it works: Each user has a single `user_settings` row. The account settings page lets the user update currency, locale, language, month start day, and monthly salary. The layout context processor reads these settings on every request and exposes them to all templates. Currency display is not hardcoded in templates; instead, display helpers convert values from INR into the chosen display currency using predefined rates.

US-3: Add income entries so monthly inflows are tracked

Where implemented: `backend/app/routes/transaction_routes.py` , `backend/app/services/transaction_service.py` , templates `transactions.html` and `home.html` .

How it works: Users create transactions with type `income` . The route validates account, category, amount, and date, then saves the transaction to MySQL. The transaction service applies the financial effect to the linked account balance, meaning income increases the account balance immediately. The dashboard then reads those transactions to compute monthly income and trends.

US-4: Add expense entries and categorize them

Where implemented: same transaction route and transaction service, plus categories in `category_routes.py` and `category_model.py`.

How it works: Expense transactions are stored with category and account references. When an expense is saved, the account balance is reduced. Categories are user-specific, so each user can maintain their own expense labels. The dashboard and reports then aggregate by category to show spending analysis, top category, and budget comparisons.

US-5: Set category budgets and get threshold alerts

Where implemented: `backend/app/routes/budget_routes.py`, `backend/app/models/budget_model.py`, `backend/app/services/notification_service.py`, dashboard and budget templates.

How it works: Budgets are monthly and category-specific. Each row stores user, category, limit amount, month, and year. When reports or notifications sync, the code calculates current expense spending per category for the current month and compares it with the saved budget. At 80% it creates a budget alert, and at 100% it creates a budget exceeded notification. The dashboard and budgets page show both the limit and actual spend.

US-6: Create savings goals and track progress

Where implemented: `backend/app/routes/goal_routes.py`, `backend/app/models/goal_model.py`, `backend/app/templates/dashboard/goals.html`.

How it works: A user can create a goal with name, target amount, current amount, deadline, and status. The goal routes support create, update, add amount, and delete. The UI shows progress visually and the dashboard surfaces the top goal separately. Deadline-based notifications are also generated when a goal is near expiry.

US-7: View charts and KPIs quickly

Where implemented: `backend/app/routes/dashboard_routes.py` with helper `_build_dashboard_data()`, and template `backend/app/templates/dashboard/home.html`.

How it works: The home dashboard calculates total balance, monthly income, monthly expense, average monthly spending, recent transactions, top spending category, top goal, budget comparison, and chart data. The monthly comparison chart is rendered from computed `monthly_stats`, and category-wise spending is rendered using a conic-gradient pie chart generated from server-side percentages.

US-8: Search, filter, and export transaction history effectively

Where implemented: `transaction_routes.py` and `report_routes.py`.

How it works: Transaction history supports filters for type, category, start date, end date, minimum amount, and maximum amount. Reports can export transaction data as CSV and full user backup as JSON. The server handles filtering, not the browser. Pagination was mentioned in the original story, but the current implementation loads the filtered result set directly without a page-number system.

US-9: Import transactions from CSV

Where implemented: `backend/app/routes/transaction_routes.py`, helper methods `_clean_import_row`, `_find_or_create_import_category`, `_find_import_account`, and `_import_transaction_exists`.

How it works: The user uploads a CSV file with required headers like date, type, category, and amount. Each row is normalized, validated, linked to an account, and assigned a category. If a transaction with the same core attributes already exists, it is skipped. This is the project's idempotency mechanism to avoid duplicate imports.

US-10: Secure access and auditable changes

Where implemented: `backend/app/utils/decorators.py`, `backend/app/utils/activity_logger.py`, `backend/app/models/activity_log_model.py`, `backend/app/services/active_session_service.py`, `backend/app/services/admin_service.py`.

How it works: Role checks distinguish admin from user. The app logs important actions such as login, logout, transaction changes, budget changes, and goal updates. Active sessions are stored separately to estimate currently logged-in users. This gives the admin

module basic auditability and system monitoring capability.

US-11: Welcome or onboarding guide

Where implemented: `backend/app/templates/auth/onboarding.html` , and the quick guide controls in `dashboard_routes.py` plus `home.html` .

How it works: There are two onboarding layers. First, the public landing page explains the product before login. Second, once the user is inside the dashboard, a quick guide panel shows how to record transactions, control repeat spending, and review patterns. The guide can be hidden and shown again using session-backed routes.

US-12: View monthly income vs expense comparison

Where implemented: dashboard home route and home template.

How it works: The server builds month-wise income and expense totals from stored transactions. These are converted into widths for bar-style visuals. The display is updated based on real transaction data every time the dashboard is loaded.

US-13: View category-wise spending analysis

Where implemented: dashboard home and reports module.

How it works: Expense transactions are grouped by category. The home screen converts the distribution into pie-chart segments and a legend, while the reports page provides a cleaner category analysis list and chart rows. The highest expense category is also highlighted on the dashboard.

US-14: View average monthly spending

Where implemented: `_build_dashboard_data()` in `dashboard_routes.py` and shown in `home.html` .

How it works: The dashboard calculates expense totals for recent months, ignores months with zero expense, and computes an average. This gives the user a simple estimate of usual monthly spending based on actual history.

US-15: Admin views account activity history

Where implemented: `admin_dashboard()` in `dashboard_routes.py` , template `admin.html` , and Mongo/SQL logging through `activity_logger.py` .

How it works: The admin dashboard shows activity rows such as login, logout, password reset, user status changes, and other actions. It also displays system transactions and usage statistics. There is a dedicated performance page for DB response, active logins, and request timing.

US-16: View budget vs actual spending

Where implemented: dashboard home, budgets route, budget template, and notification service.

How it works: For every saved budget in the current month, the code calculates actual expense spending in that category and computes percentage used. It then marks values visually and generates alert notifications when thresholds are crossed.

US-17: Edit or delete transactions

Where implemented: `edit_transaction()` and `delete_transaction()` in `transaction_routes.py` .

How it works: Editing a transaction first reverses the old financial effect from the old account, applies the new values, and then writes the updated transaction. Deleting a transaction similarly reverses its effect from the account balance so totals remain consistent.

US-18: Add notes or descriptions to transactions

Where implemented: transaction forms and transaction model.

How it works: The transaction table includes a `description` field. Add, edit, CSV import, dashboard recent-transaction view, and reports all preserve and display this description.

US-19: Mark notifications as read

Where implemented: `notification_routes.py`, `notification_model.py`, and `notifications.html`.

How it works: Notifications are stored in MySQL with an `is_read` flag. The user can mark a single notification as read or mark all as read. The header bell icon uses the unread count injected from the context processor.

US-20: View transaction history for a selected date range

Where implemented: `transaction_list()` in `transaction_routes.py`.

How it works: The history screen accepts start-date and end-date query parameters and applies them directly in the database query. This lets the user narrow down history to a meaningful period for review or export.

Part B: Module Wise Explanation

Module 1: Authentication and Authorization

Main files: `auth_routes.py` , `auth.html` , `user_model.py` , `decorators.py` , `admin_service.py` .

Purpose: Handles registration, login, logout, recovery, account blocking, and role-based separation between normal users and admin.

Key implementation details:

- Username and password are validated using regex rules before user creation.
- Password hashes are created with Werkzeug, not saved in plain text.
- Security-question answers are normalized and hashed before storage.
- Login attempts are rate-limited through Flask session variables.
- Blocked users are denied login using the `is_active` field.
- Admin is identified by role and default seeded email.

Likely evaluator question: ?How is password reset handled?? Answer: the current implementation uses stored hashed security-question answers instead of mail-token reset.

Module 2: Transaction Management

Main files: `transaction_routes.py` , `transaction_model.py` , `transaction_service.py` , `transactions.html` .

Purpose: Store all income and expense records and keep account balances consistent.

- Add transaction validates account/category ownership by user.
- Edit transaction reverses previous balance effect before applying the new one.
- Delete transaction also reverses financial effect.
- Filtering supports date range, type, category, and amount bounds.
- CSV import supports deduplication and category/account inference.

Design reason: balance is not recalculated from scratch each time; it is maintained incrementally for performance and immediate UI updates.

Module 3: Category Management

Main files: `category_routes.py` , `category_model.py` , `categories.html` .

Purpose: Give each user a controlled set of income and expense categories.

- Default categories are created for each user through `ensure_default_categories()` .
- User-defined categories are stored per user, so categories are isolated between users.
- Both transaction entry and CSV import depend on valid categories.

Module 4: Dashboard Module

Main files: `dashboard_routes.py` , `home.html` , `layout.html` , `dashboard.css` .

Purpose: Present a quick financial summary with balance, trends, category analysis, budgets, goals, and recent activity.

- Data is built server-side in one helper function so the dashboard remains consistent.
- Summary cards show total balance, monthly income, monthly expense, and average monthly spending.
- Monthly comparison uses computed widths for income and expense bars.
- Category-wise spending uses conic-gradient based pie chart styling.
- The dashboard also surfaces top category, top goal, recent transactions, and budget-vs-actual cards.

Module 5: Budget Planning

Main files: `budget_routes.py` , `budget_model.py` , `budgets.html` , `notification_service.py` .

Purpose: Let users define spending ceilings per category for each month.

- Budgets are stored per user, month, year, and category.
- Actual monthly spend is calculated from real expense transactions.
- Budget usage percentage, remaining amount, and alert state are displayed in UI.
- Recurring expenses are managed from the budget page because they influence future spending control.

Module 6: Reports and Analytics

Main files: `report_routes.py` , `report_service.py` , `reports.html` .

Purpose: Provide deeper analysis and export facilities beyond the main dashboard.

- Filters support date range, category, and type.
- Outputs include total income, total expense, net balance, category analysis, and monthly summary.
- Exports include CSV for transactions and JSON for full backup.
- MongoDB report snapshots are displayed to show report history / analytic capture.

Module 7: Reminder and Notifications

Main files: `notification_model.py` , `notification_service.py` , `notification_routes.py` , `notifications.html` .

Purpose: Inform the user about welcome state, budget thresholds, and goal deadlines.

- Notifications are generated programmatically by `sync_notifications()` .
- The service avoids duplicates using `reference_key` .
- Notifications can be marked read individually or all at once.
- The header bell shows unread count on every page.

Module 8: Data Export and Backup

Main files: `report_routes.py` , `reports.html` .

Purpose: Allow the user to take data out of the system for storage or external review.

- Transactions can be downloaded as CSV for spreadsheet use.
- User data can be downloaded as JSON backup.
- CSV import is also supported from the transaction module, giving two-way movement of data.

Module 9: Saving and Goals

Main files: `goal_routes.py` , `goal_model.py` , `goals.html` .

Purpose: Help users plan savings targets and track progress over time.

- Each goal has name, target, current progress, deadline, and status.
- There is a direct add-amount action so users can increment progress without rewriting the whole goal.
- The dashboard highlights the most important goal for quick visibility.

Module 10: Account Management

Main files: `account_model.py` , `dashboard_routes.py` , `account.html` , `layout.html` .

Purpose: Manage profile information, display settings, password changes, and financial accounts used in transactions.

- Users can update name, settings, and password.
- Users can add/delete financial accounts such as Cash, Bank, or UPI.
- Default account creation is automated for smoother onboarding.
- The account module is reached from the profile dropdown in the header, not from a separate public auth screen.

Module 11: Admin and System Module

Main files: `dashboard_routes.py` , `admin.html` , `admin_performance.html` , `active_session_service.py` , `activity_logger.py` , `admin_service.py` .

Purpose: Give the single system admin visibility into users, activity, system transactions, login state, and performance indicators.

- Admin can see all users and whether they are active or blocked.
- Admin can block/unblock or delete users.
- Admin can view system-wide transactions and activity logs.
- Admin performance page shows active logins, DB response time, and last request time.
- Default admin account is seeded automatically if not present.

Supporting Infrastructure

Schema Migration Safety

Where implemented: `schema_service.py` .

This service checks whether important columns and tables exist and creates or alters them if missing. This is important because the project evolved over time and older databases may not contain the new columns needed for security answers, language, salary automation, or recurring transactions.

Currency Conversion

Where implemented: `currency_service.py` .

Amounts are conceptually treated in INR and then converted for display using stored rates. That is why templates use display helpers rather than embedding hardcoded currency symbols everywhere.

Language Support

Where implemented: `i18n_service.py` and the layout context processor.

The app normalizes language codes to English, Hindi, or Marathi. This is not a full translation engine, but it lays the groundwork for multilingual display logic and configuration-aware UI.

MongoDB Usage

Where used: activity logs and report snapshots.

MySQL remains the primary application database. MongoDB is used for flexible document-style storage of logs and snapshots where schema is less rigid and historical/event-style data is useful.

Evaluator Question Prep

Likely Question	Strong Answer
How do balances change when transactions are edited or deleted?	The project uses an incremental account-balance model. It first reverses the old transaction effect from the account, then applies the new effect, so totals remain accurate without recalculating every transaction each time.
How do you prevent duplicate imported transactions?	Before inserting a CSV row, the code checks whether a transaction with the same user, account, category, type, amount, date, and description already exists. If yes, it skips that row.
How are budget alerts generated?	The notification service compares current-month expense totals against saved monthly budgets. At 80% it creates a warning; at 100% it creates an exceeded alert.
Why use MongoDB when MySQL already exists?	MySQL stores the structured finance records. MongoDB is used for log-style and snapshot-style data where flexible schema and document storage are useful, such as activity logs and saved report snapshots.

How is admin access controlled?	Admin is role-based and checked through decorators. The admin dashboard and performance routes are protected so normal users cannot open them.
What automation features exist?	There are two main ones: automatic monthly salary posting and recurring transactions. These are executed through automation services and saved recurring-transaction records.
What is the difference between dashboard and reports?	Dashboard is the quick summary screen for everyday use. Reports is the deeper analysis/export area with filters, category analysis, monthly summary, CSV export, and JSON backup.

Final Summary

The project is not just a CRUD app. The latest implementation includes secure auth, account-aware transaction handling, monthly budgeting, savings goals, activity auditing, notifications, CSV import/export, JSON backup, Mongo-backed snapshots/logs, recurring transaction automation, salary automation, and admin/system monitoring. For teammates preparing for evaluation, the safest explanation is to describe the app as a Flask finance platform with MySQL as the primary transactional database, MongoDB as the log/snapshot store, and a layered structure of models, routes, services, templates, and shared utilities.